

LAB11.1

DATA STRUCTURES WITH AI

Implementing Fundamental Structures

Submitted By

Name: Bitla Sairaj

Roll Number: 2503a52162

Branch: CSE ai&ml

Section: 06

Date: 10-09-2026

1.INTRODUCTION

This lab focuses on implementing fundamental data structures in Python with the help of AI tools. It covers eight core data structures — Stack, Queue, Linked List, Binary Search Tree, Hash Table, Graph, Priority Queue, and Deque — followed by two real-world scenarios (a Campus Resource Management System and a Smart E-Commerce Platform) where an appropriate data structure must be selected and justified for each feature, with one feature implemented per scenario.

For each data structure, AI was used to help generate the class implementation with proper docstrings, which was then tested with sample data to confirm correct behavior before being documented in this report.

2.TOOLS USED

- Python
- Cursor AI / GitHub Copilot
- VSCode
- Terminal

3.TASK 1—STACK

3.1 Objective

Use AI to generate a Stack class with `push()`, `pop()`, `peek()`, and `is_empty()` methods, with proper docstrings.

3.2 Implementation

```
class Stack:
    """A simple stack implementation using a Python list."""

    def __init__(self):
        self.items = []

    def push(self, item):
        """Add an item to the top of the stack."""
        self.items.append(item)

    def pop(self):
        """Remove and return the top item from the stack."""
        if self.is_empty():
            return None
        return self.items.pop()

    def peek(self):
        """Return the top item without removing it."""
        if self.is_empty():
            return None
```

```
return self.items[-1]

def is_empty(self):
    """Return True if the stack is empty."""
    return len(self.items) == 0
```

3.3 Test Code

```
stack = Stack()

stack.push(10)
stack.push(20)
stack.push(30)

print("Stack:", stack.items)
print("Top element:", stack.peek())
print("Popped:", stack.pop())
print("Stack after pop:", stack.items)
print("Is empty?", stack.is_empty())
```

3.4 Expected Output

```
Stack: [10, 20, 30]
Top element: 30
Popped: 30
Stack after pop: [10, 20]
Is empty? False
```

3.5 Notes

A stack follows LIFO (Last In, First Out) order — the last element pushed is the first one popped.

3.6 Screenshots

```
1 class Stack:
2     def __init__(self):
3         self.items = []
4
5     def push(self, item):
6         self.items.append(item)
7
8     def pop(self):
9         if self.is_empty():
10            return None
11        return self.items.pop()
12
13    def peek(self):
14        if self.is_empty():
15            return None
16        return self.items[-1]
17
18    def is_empty(self):
19        return len(self.items) == 0
20
21 stack = Stack()
22
23 stack.push(10)
24 stack.push(20)
25 stack.push(30)
26
27 print("Stack:", stack.items)
28 print("Top element:", stack.peek())
29 print("Popped:", stack.pop())
30 print("Stack after pop:", stack.items)
31 print("Is empty?", stack.is_empty())
```

```
PS D:\sairaj course> & "C:\Program Files\Python314\python.exe" "d:/sairaj course/Lab_11_Data_Structures.py"
Stack: [10, 20, 30]
Top element: 30
Popped: 30
Stack after pop: [10, 20]
Is empty? False
PS D:\sairaj course> |
```

4.TASK2—QUEUE

4.1 Objective

Use AI to generate a Queue class using a Python list, with enqueue(), dequeue(), peek(), and size() methods that demonstrate FIFO behavior.

4.2 Implementation

```
class Queue:
    """A simple FIFO queue using a Python list."""

    def __init__(self):
        self.items = []

    def enqueue(self, item):
        """Add an item to the rear of the queue."""
        self.items.append(item)

    def dequeue(self):
        """Remove and return the front item."""
        if self.size() == 0:
            return None
        return self.items.pop(0)

    def peek(self):
        """Return the front item without removing it."""
        if self.size() == 0:
            return None
        return self.items[0]

    def size(self):
        """Return the number of items in the queue."""
        return len(self.items)
```

4.3 Test Code

```
queue = Queue()

queue.enqueue("Student 1")
queue.enqueue("Student 2")
queue.enqueue("Student 3")

print("Queue:", queue.items)
print("Front:", queue.peek())
print("Dequeued:", queue.dequeue())
print("Queue after dequeue:", queue.items)
print("Size:", queue.size())
```

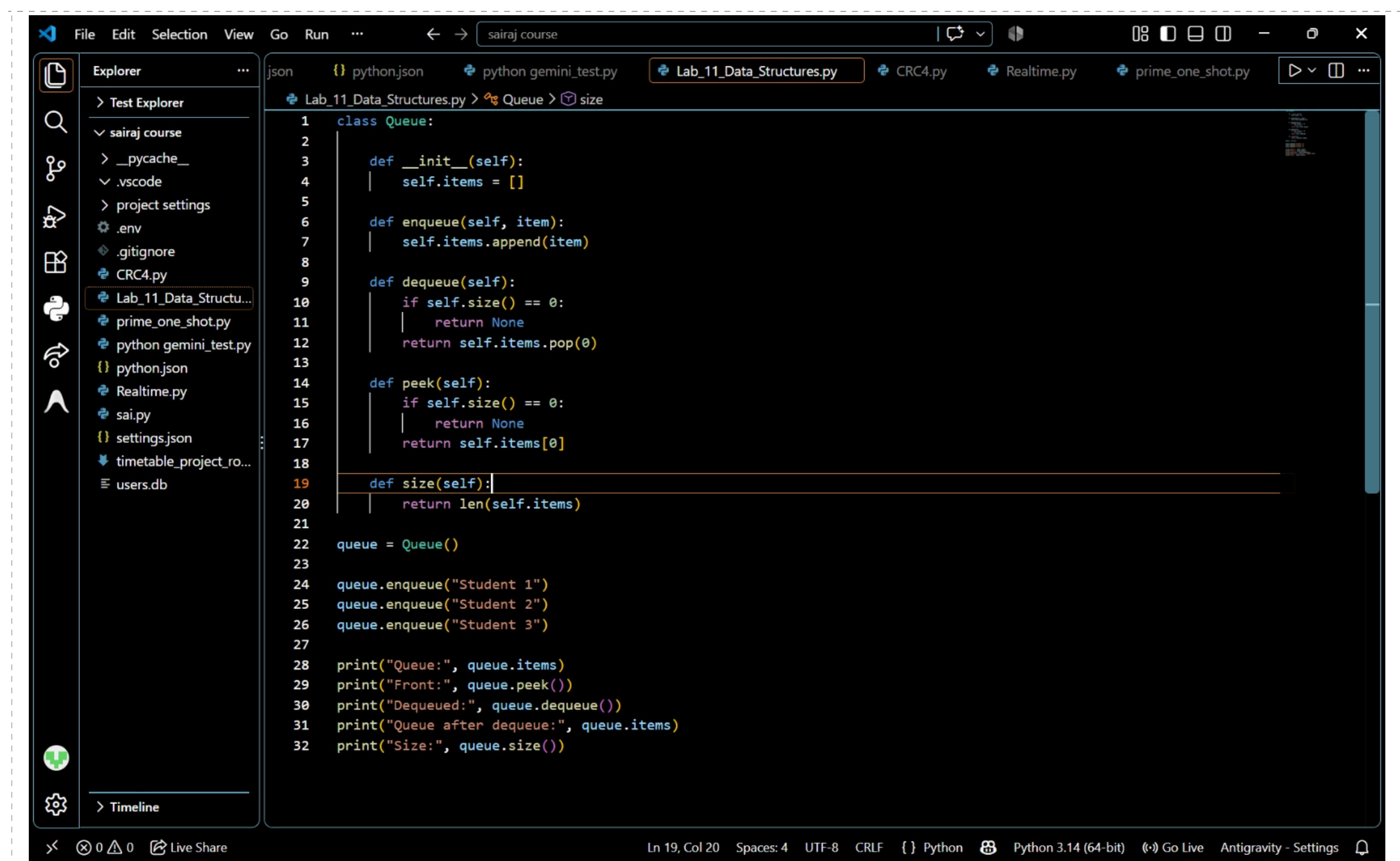
4.4 Expected Output

```
Queue: ['Student 1', 'Student 2', 'Student 3']  
Front: Student 1  
Dequeued: Student 1  
Queue after dequeue: ['Student 2', 'Student 3']  
Size: 2
```

4.5 Notes

A queue follows FIFO (First In, First Out) order — the first element enqueued is the first one dequeued.

4.6 Screenshots

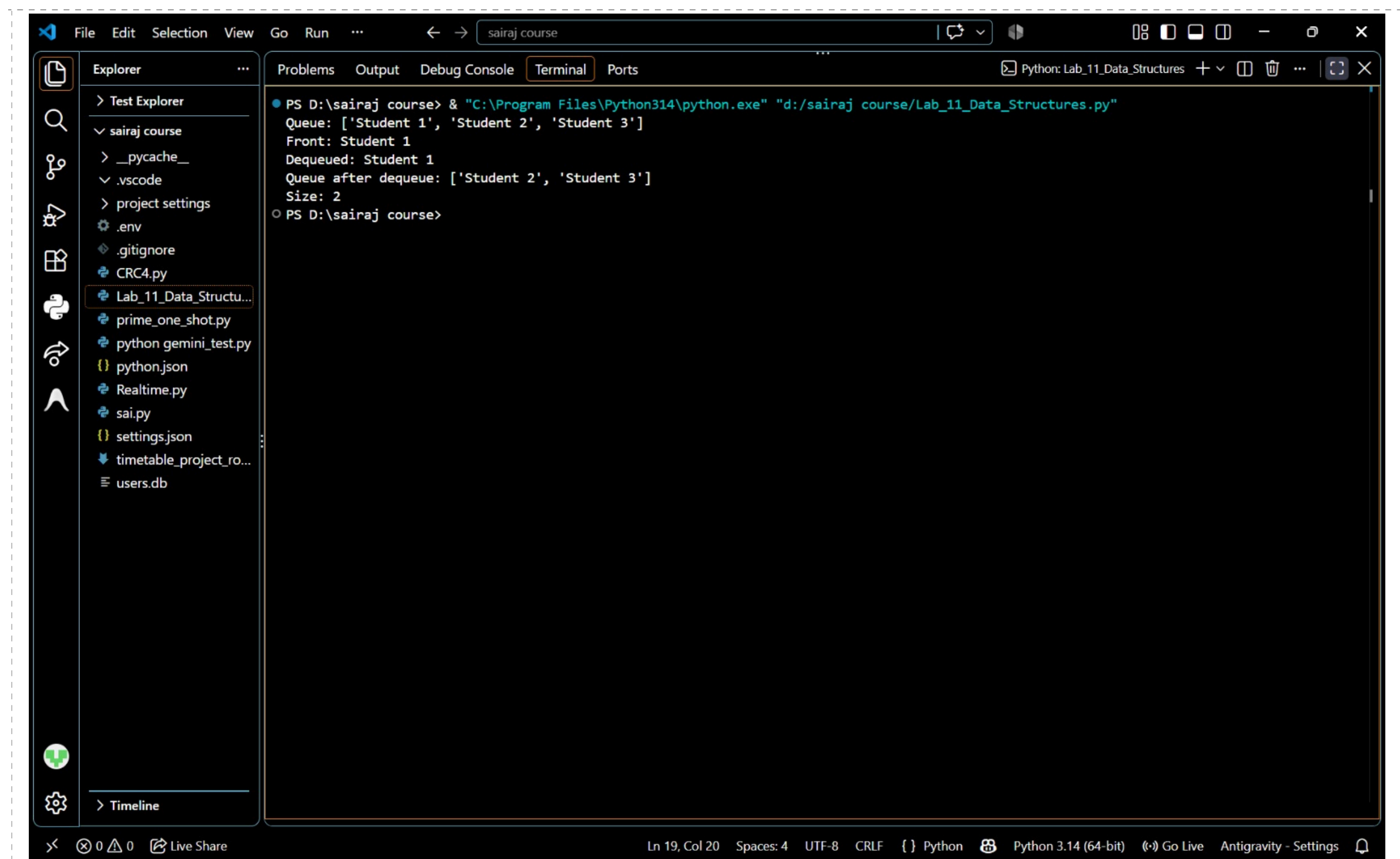


The screenshot shows a VS Code editor window with the file explorer on the left and a Python file named `Lab_11_Data_Structures.py` open in the editor. The code implements a queue class and demonstrates its usage.

```
1 class Queue:
2
3     def __init__(self):
4         self.items = []
5
6     def enqueue(self, item):
7         self.items.append(item)
8
9     def dequeue(self):
10        if self.size() == 0:
11            return None
12        return self.items.pop(0)
13
14    def peek(self):
15        if self.size() == 0:
16            return None
17        return self.items[0]
18
19    def size(self):
20        return len(self.items)
21
22 queue = Queue()
23
24 queue.enqueue("Student 1")
25 queue.enqueue("Student 2")
26 queue.enqueue("Student 3")
27
28 print("Queue:", queue.items)
29 print("Front:", queue.peek())
30 print("Dequeued:", queue.dequeue())
31 print("Queue after dequeue:", queue.items)
32 print("Size:", queue.size())
```

The output of the code, as shown in the terminal, is:

```
Queue: ['Student 1', 'Student 2', 'Student 3']  
Front: Student 1  
Dequeued: Student 1  
Queue after dequeue: ['Student 2', 'Student 3']  
Size: 2
```



5. TASK 3—SINGLY LINKED LIST

5.1 Objective

Use AI to generate a singly linked list with a Node class, a LinkedList class, an insert() method, and a display() method, with clear documentation.

5.2 Implementation

```
class Node:
    """Represents a node in a singly linked list."""

    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    """A singly linked list implementation."""

    def __init__(self):
        self.head = None

    def insert(self, data):
```

```
"""Insert a new node at the end of the linked list."""
new_node = Node(data)

if self.head is None:
    self.head = new_node
    return

current = self.head

while current.next:
    current = current.next

current.next = new_node

def display(self):
    """Display all elements of the linked list."""
    current = self.head

    while current:
        print(current.data, end=" -> ")
        current = current.next

    print("None")
```

5.3 Test Code

```
linked_list = LinkedList()

linked_list.insert(10)
linked_list.insert(20)
linked_list.insert(30)

linked_list.display()
```

5.4 Expected Output

```
10 -> 20 -> 30 -> None
```

5.5 Screenshots

```
1 class Node:
2     def __init__(self, data):
3         self.data = data
4         self.next = None
5
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9     def insert(self, data):
10        new_node = Node(data)
11
12        if self.head is None:
13            self.head = new_node
14            return
15
16        current = self.head
17
18        while current.next:
19            current = current.next
20
21        current.next = new_node
22    def display(self):
23        current = self.head
24        while current:
25            print(current.data, end=" -> ")
26            current = current.next
27        print("None")
28
29 linked_list = LinkedList()
30
31 linked_list.insert(10)
32 linked_list.insert(20)
33 linked_list.insert(30)
34
35 linked_list.display()
```

```
PS D:\sairaj course> & "C:\Program Files\Python314\python.exe" "d:/sairaj course/Lab_11_Data_Structures.py"
10 -> 20 -> 30 -> None
PS D:\sairaj course>
```

6.TASK 4—BINARY SEARCH TREE

6.1 Objective

Use AI to generate a Binary Search Tree with a recursive insert() method and an in-order traversal method.

6.2 Implementation

```
class BSTNode:
    """Represents a node in a Binary Search Tree."""

    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BST:
    """Binary Search Tree implementation."""

    def __init__(self):
        self.root = None

    def insert(self, data):
        """Insert data into the BST."""
        self.root = self._insert_recursive(self.root, data)

    def _insert_recursive(self, root, data):
        """Recursively insert data into the correct position."""
        if root is None:
            return BSTNode(data)

        if data < root.data:
            root.left = self._insert_recursive(root.left, data)
        elif data > root.data:
            root.right = self._insert_recursive(root.right, data)

        return root

    def inorder(self):
        """Perform in-order traversal of the BST."""
        self._inorder_recursive(self.root)

    def _inorder_recursive(self, root):
        """Recursively perform in-order traversal."""
        if root:
            self._inorder_recursive(root.left)
            print(root.data, end=" ")
            self._inorder_recursive(root.right)
```

6.3 Test Code

```
bst=BST()

for value in [50,30,70,20,40,60,80]:
    bst.insert(value)

print("In-order traversal:")
bst.inorder()
```

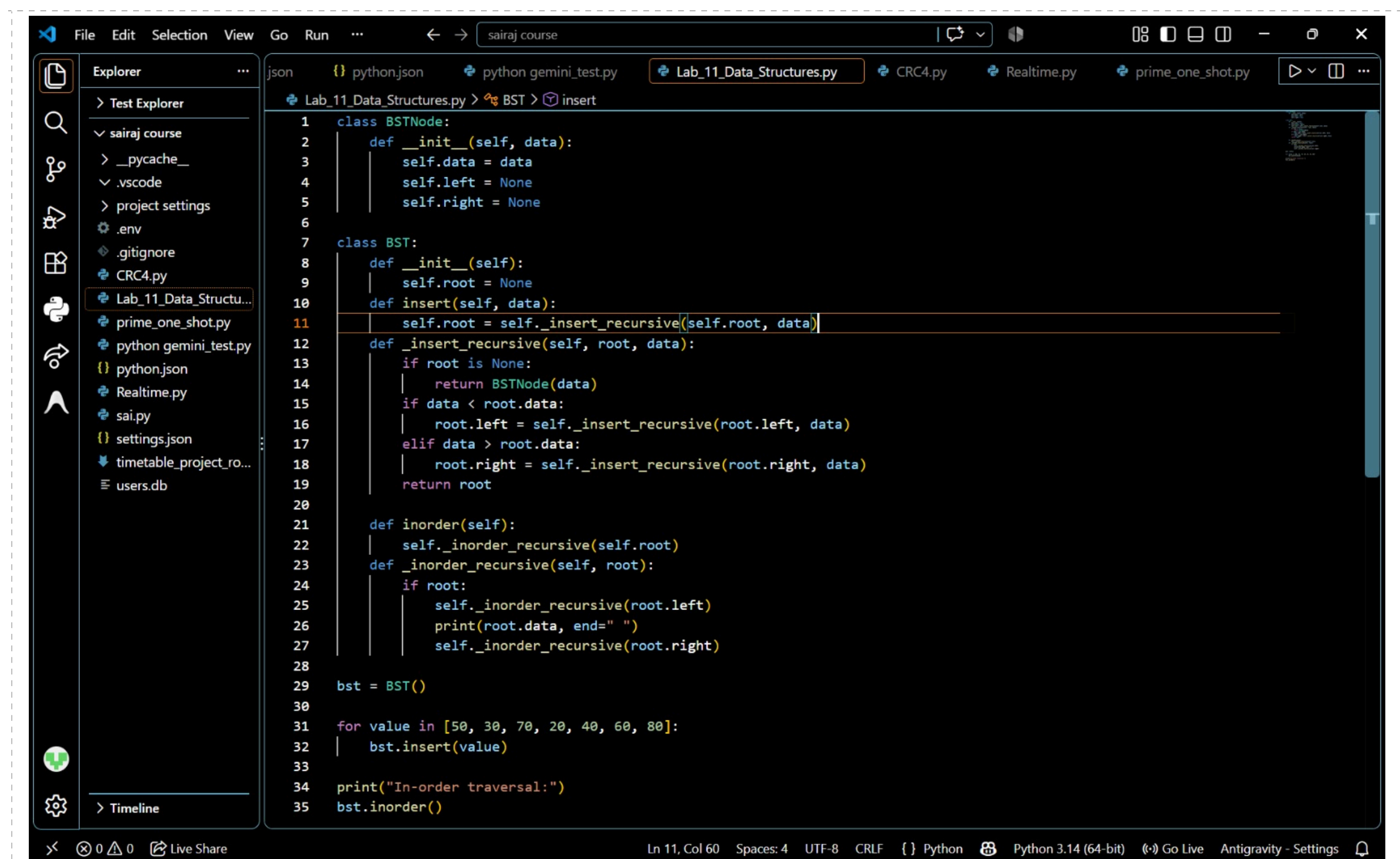
6.4 Expected Output

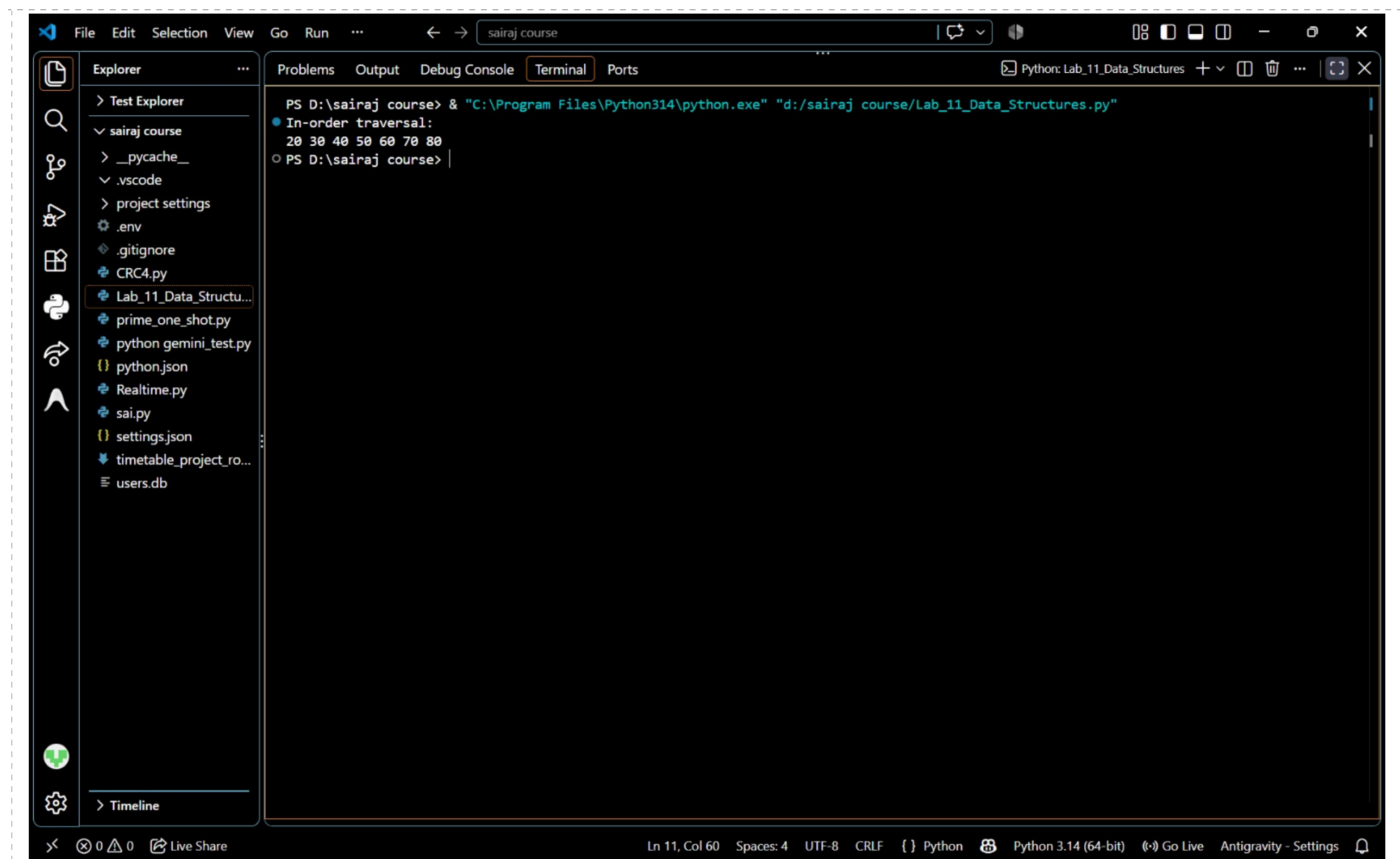
```
In-order traversal:
20 30 40 50 60 70 80
```

6.5 Notes

In-order traversal of a BST always produces the values in sorted order, which is a useful property to point out in the explanation.

6.6 Screenshots





7. TASK 5—HASH TABLE

7.1 Objective

Use AI to generate a Hash Table with insert(), search(), and delete() methods, handling collisions using chaining.

7.2 Implementation

```
class HashTable:
    """Hash table implementation using chaining for collisions."""

    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate an index for the given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert or update a key-value pair."""
        index = self._hash(key)
```

```

        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return

        self.table[index].append([key, value])

    def search(self, key):
        """Search for a value using its key."""
        index = self._hash(key)

        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]

        return None

    def delete(self, key):
        """Delete a key-value pair."""
        index = self._hash(key)

        for pair in self.table[index]:
            if pair[0] == key:
                self.table[index].remove(pair)
                return True

        return False

```

7.3 Test Code

```

ht = HashTable()

ht.insert(101, "Sai")
ht.insert(102, "Rahul")
ht.insert(103, "Kiran")

print("Search 102:", ht.search(102))

ht.delete(102)

print("Search 102 after deletion:", ht.search(102))

```

7.4 Expected Output

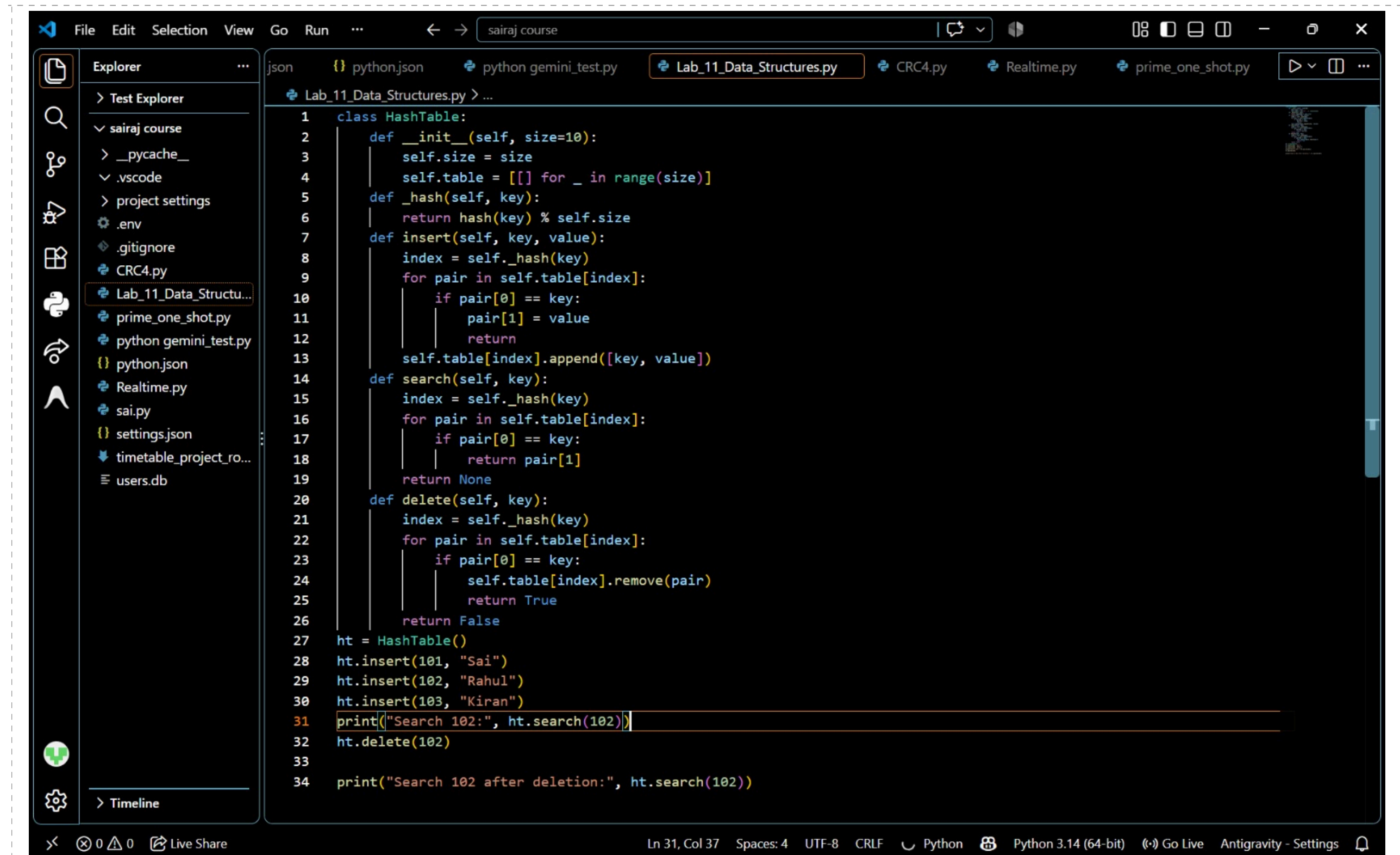
```
Search 102: Rahul
```

Search 102 after deletion: None

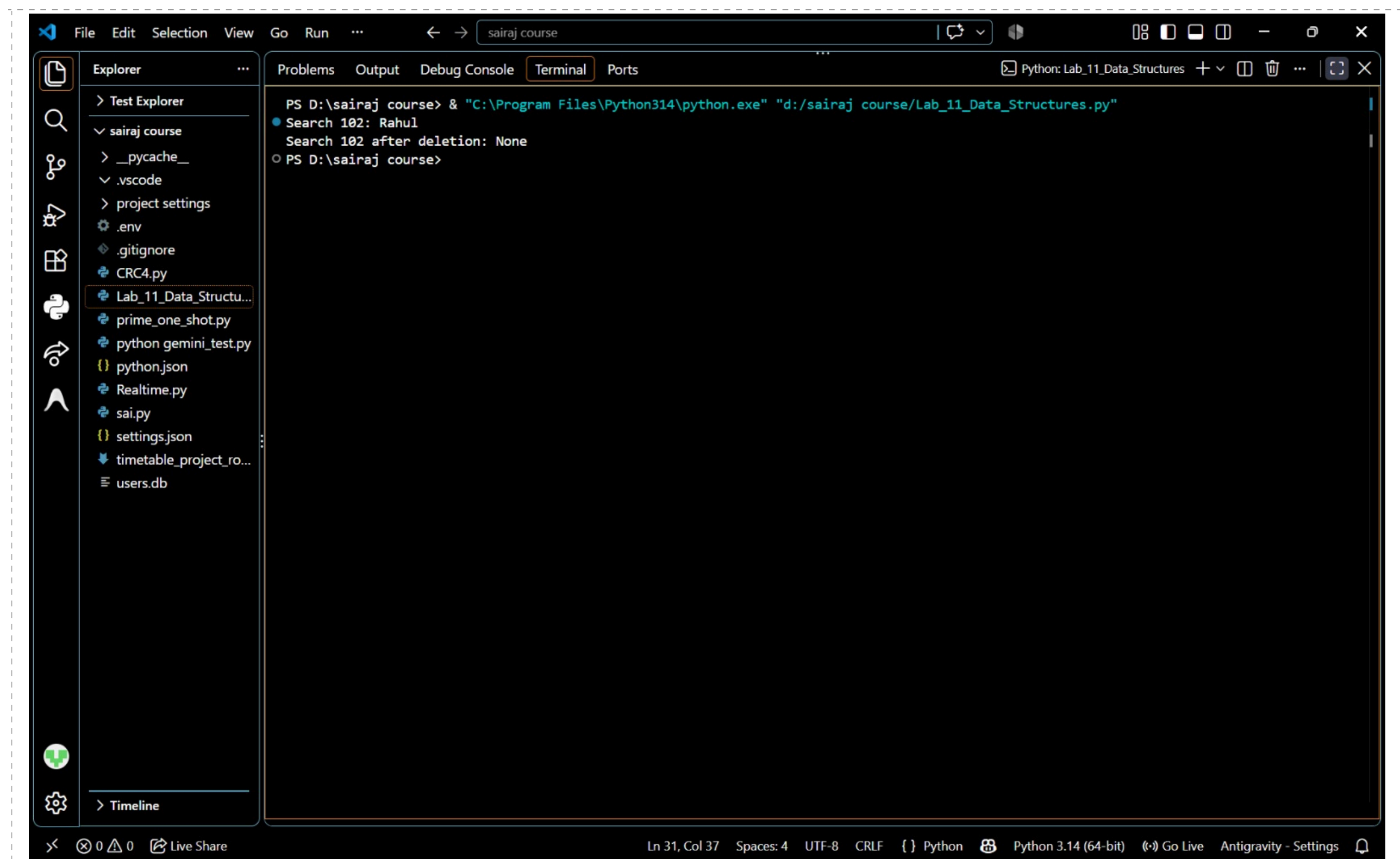
7.5 Notes

Chaining handles collisions by storing multiple key-value pairs in a list at the same index, so entries that hash to the same slot don't overwrite each other.

7.6 Screenshots



```
1 class HashTable:
2     def __init__(self, size=10):
3         self.size = size
4         self.table = [[] for _ in range(size)]
5     def _hash(self, key):
6         return hash(key) % self.size
7     def insert(self, key, value):
8         index = self._hash(key)
9         for pair in self.table[index]:
10            if pair[0] == key:
11                pair[1] = value
12                return
13        self.table[index].append([key, value])
14    def search(self, key):
15        index = self._hash(key)
16        for pair in self.table[index]:
17            if pair[0] == key:
18                return pair[1]
19        return None
20    def delete(self, key):
21        index = self._hash(key)
22        for pair in self.table[index]:
23            if pair[0] == key:
24                self.table[index].remove(pair)
25                return True
26        return False
27
28 ht = HashTable()
29 ht.insert(101, "Sai")
30 ht.insert(102, "Rahul")
31 ht.insert(103, "Kiran")
32 print("Search 102:", ht.search(102))
33 ht.delete(102)
34 print("Search 102 after deletion:", ht.search(102))
```



8.TASK 6—GRAPH

8.1Objective

Use AI to generate a Graph implementation using an adjacency list, with methods to add vertices, add edges, and display connections.

8.2Implementation

```
class Graph:
    """Graph implementation using an adjacency list."""

    def __init__(self):
        self.graph = {}

    def add_vertex(self, vertex):
        """Add a vertex to the graph."""
        if vertex not in self.graph:
            self.graph[vertex] = []

    def add_edge(self, vertex1, vertex2):
        """Create an edge between two vertices."""
        self.add_vertex(vertex1)
        self.add_vertex(vertex2)
```

```
self.graph[vertex1].append(vertex2)

def display(self):
    """Display all vertex connections."""
    for vertex, connections in self.graph.items():
        print(vertex, "->", connections)
```

8.3 Test Code

```
graph = Graph()

graph.add_edge("College", "Library")
graph.add_edge("College", "Cafeteria")
graph.add_edge("Library", "Bus Stop")

graph.display()
```

8.4 Expected Output

```
College->['Library','Cafeteria']
Library->['Bus Stop']
Cafeteria-> []
BusStop->[]
```

8.5 Screenshots

```
1 class Graph:
2     def __init__(self):
3         self.graph = {}
4
5     def add_vertex(self, vertex):
6         if vertex not in self.graph:
7             self.graph[vertex] = []
8
9     def add_edge(self, vertex1, vertex2):
10        self.add_vertex(vertex1)
11        self.add_vertex(vertex2)
12
13        self.graph[vertex1].append(vertex2)
14
15    def display(self):
16        for vertex, connections in self.graph.items():
17            print(vertex, "->", connections)
18
19 graph = Graph()
20
21 graph.add_edge("College", "Library")
22 graph.add_edge("College", "Cafeteria")
23 graph.add_edge("Library", "Bus Stop")
24
25 graph.display()
```

```
PS D:\sairaj course> & "C:\Program Files\Python314\python.exe" "d:/sairaj course/Lab_11_Data_Structures.py"
College -> ['Library', 'Cafeteria']
Library -> ['Bus Stop']
Cafeteria -> []
Bus Stop -> []
PS D:\sairaj course>
```

9.TASK 7—PRIORITY QUEUE

9.1 Objective

Use AI to generate a Priority Queue using Python's heapq module, with enqueue (with priority), dequeue (highest priority), and display methods.

9.2 Implementation

```
import heapq

class PriorityQueue:
    """Priority queue implementation using heapq."""

    def __init__(self):
        self.queue = []

    def enqueue(self, item, priority):
        """Add an item with a priority."""
        heapq.heappush(self.queue, (priority, item))

    def dequeue(self):
        """Remove and return the highest-priority item."""
        if not self.queue:
            return None

        priority, item = heapq.heappop(self.queue)
        return item

    def display(self):
        """Display the priority queue."""
        print(self.queue)
```

9.3 Test Code

```
pq = PriorityQueue()

pq.enqueue("Normal Task", 3)
pq.enqueue("Important Task", 2)
pq.enqueue("Emergency Task", 1)

pq.display()

print("Dequeued:", pq.dequeue())
```

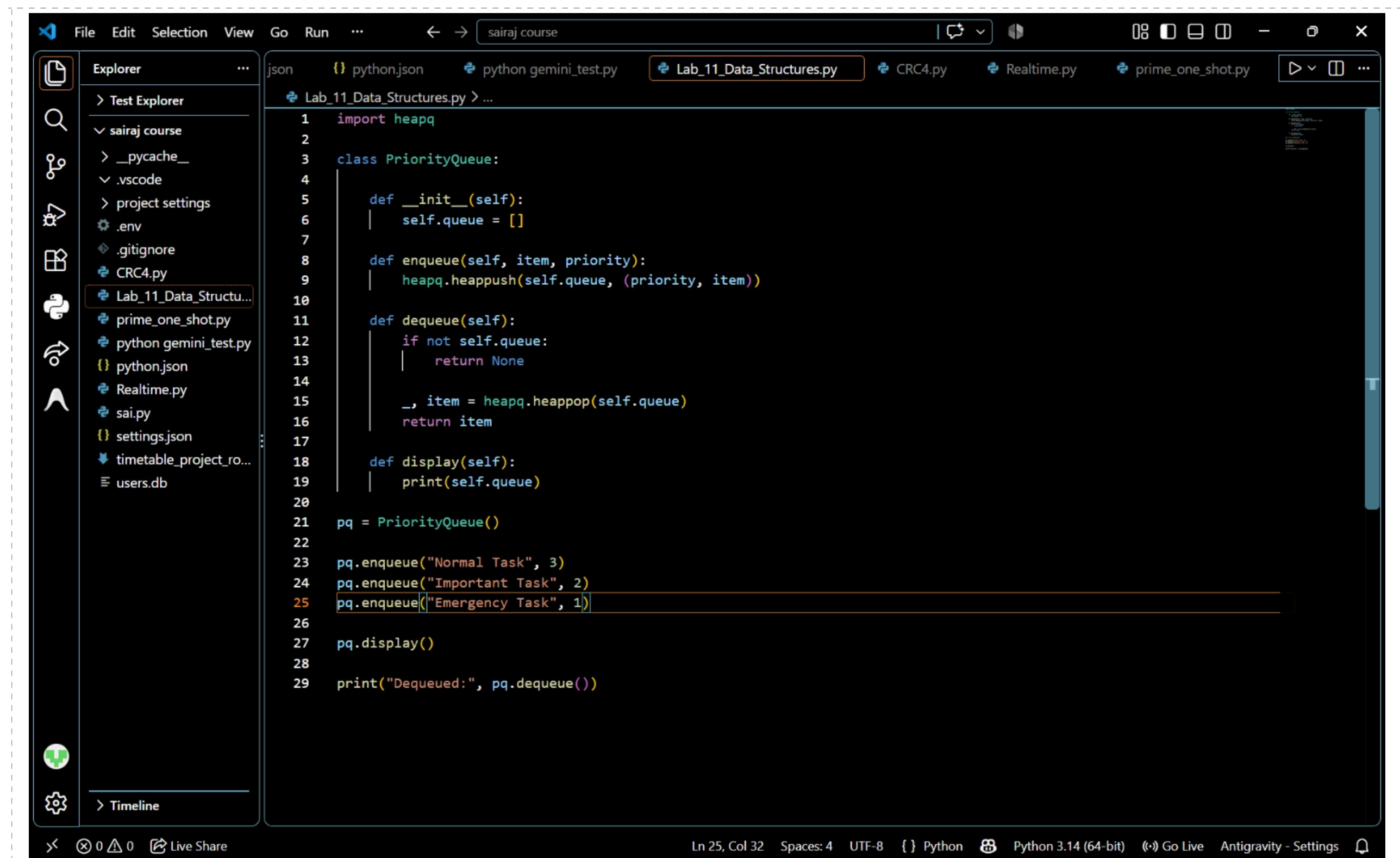
9.4 Expected Output

Dequeued: Emergency Task

9.5 Notes

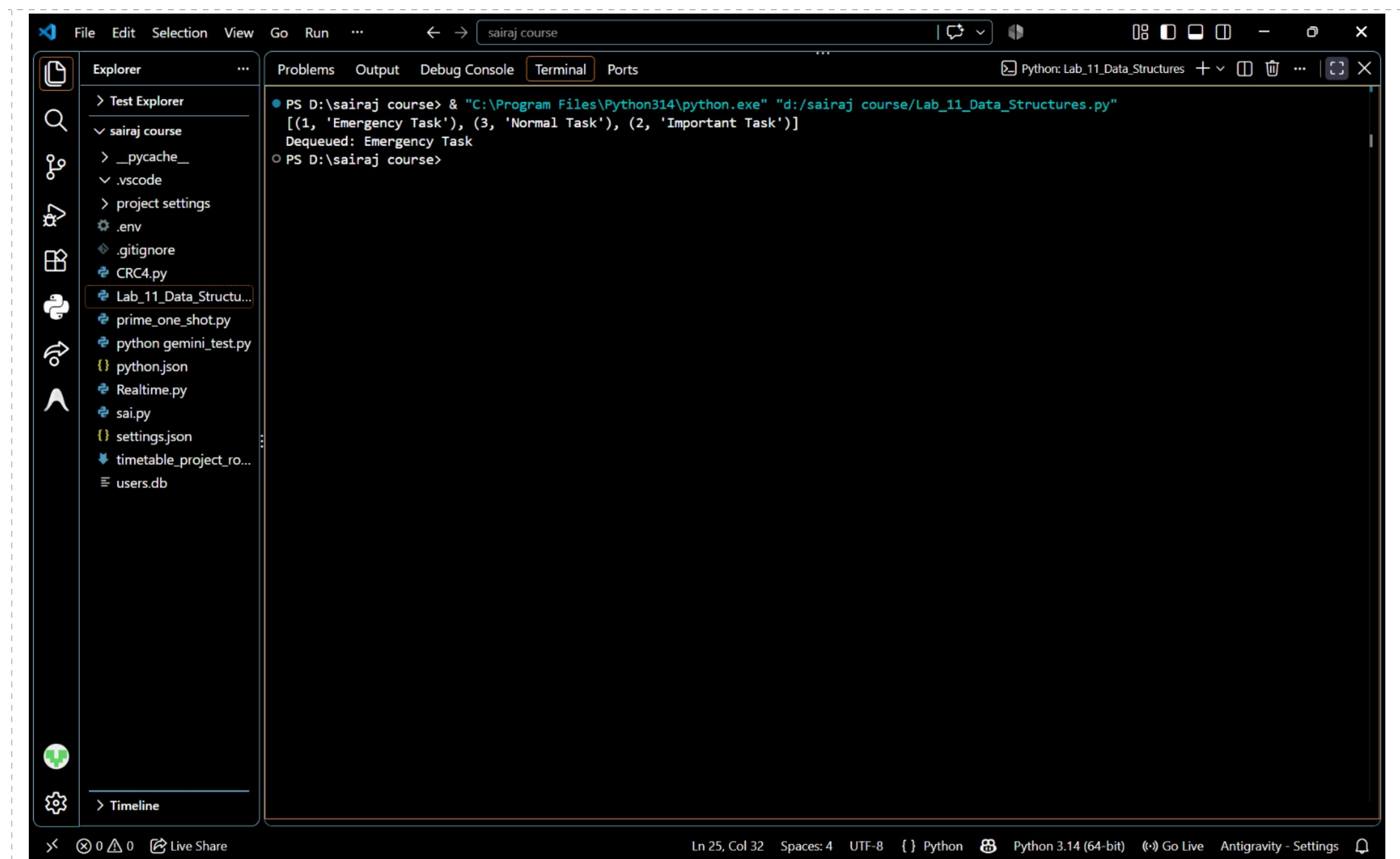
Python's `heapq` is a min-heap, so this implementation treats a smaller priority number as a higher priority (e.g. priority 1 is served before priority 3).

9.6 Screenshots



The screenshot shows a VS Code editor window with the file `Lab_11_Data_Structures.py` open. The code implements a `PriorityQueue` class using Python's `heapq` module. The class has three methods: `__init__` to initialize an empty queue, `enqueue` to add items with a priority, and `dequeue` to remove and return the item with the highest priority (lowest number). A `display` method is also present to show the current state of the queue. The script creates a `PriorityQueue` instance, enqueues three tasks with priorities 3, 2, and 1, displays the queue, and then dequeues the highest priority task, which is 'Emergency Task'.

```
1 import heapq
2
3 class PriorityQueue:
4
5     def __init__(self):
6         self.queue = []
7
8     def enqueue(self, item, priority):
9         heapq.heappush(self.queue, (priority, item))
10
11    def dequeue(self):
12        if not self.queue:
13            return None
14
15        _, item = heapq.heappop(self.queue)
16        return item
17
18    def display(self):
19        print(self.queue)
20
21 pq = PriorityQueue()
22
23 pq.enqueue("Normal Task", 3)
24 pq.enqueue("Important Task", 2)
25 pq.enqueue("Emergency Task", 1)
26
27 pq.display()
28
29 print("Dequeued:", pq.dequeue())
```



10. TASK 8—DEQUE

10.1 Objective

Use AI to generate a Deque implementation using `collections.deque`, supporting insertion and removal from both the front and rear.

10.2 Implementation

```
from collections import deque

class DequeDS:
    """Double-ended queue using collections.deque."""

    def __init__(self):
        self.deque = deque()

    def add_front(self, item):
        """Insert an item at the front."""
        self.deque.appendleft(item)

    def add_rear(self, item):
        """Insert an item at the rear."""
```

```
self.deque.append(item)

def remove_front(self):
    """Remove and return the front item."""
    if not self.deque:
        return None
    return self.deque.popleft()

def remove_rear(self):
    """Remove and return the rear item."""
    if not self.deque:
        return None
    return self.deque.pop()
```

10.3 Test Code

```
dq = DequeDS()

dq.add_front(10)
dq.add_rear(20)
dq.add_front(5)

print("Deque:", dq.deque)

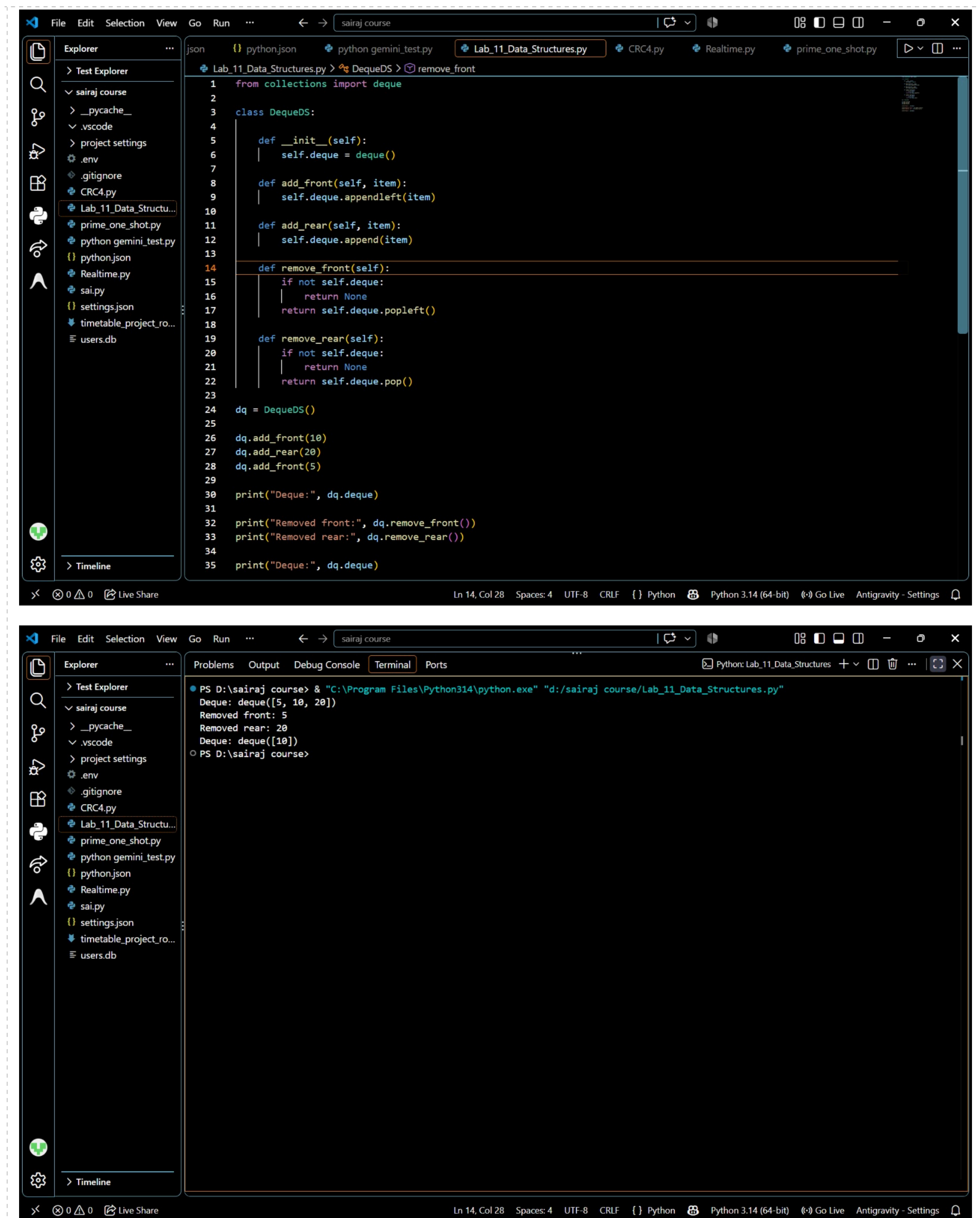
print("Removed front:", dq.remove_front())
print("Removed rear:", dq.remove_rear())

print("Deque:", dq.deque)
```

10.4 Expected Output

```
Deque: deque([5, 10, 20])
Removed front: 5
Removed rear: 20
Deque: deque([10])
```

10.5 Screenshots



11. TASK 9—CAMPUS RESOURCE MANAGEMENT SYSTEM

11.1 Objective

Select an appropriate data structure for each of five features of a Campus Resource Management System, justify each choice in 2– 3 sentences, and implement one feature.

11.2 Feature– Data Structure Selection

Feature	Recommended Data Structure	Justification
Student Attendance Tracking	Hash Table	A hash table provides fast lookup of student records using a student ID, which is suitable when we need to quickly check whether a particular student has an attendance record.
Event Registration System	Hash Table	Participants can be identified using a unique student/registration ID. Hash-table lookup allows quick searching and removal of a participant.
Library Book Borrowing	Priority Queue	Books can be organized according to due dates, allowing the system to prioritize books whose return dates are approaching.
Bus Scheduling System	Graph	Bus stops and routes naturally form a network. Vertices can represent stops while edges represent connections between stops.
Cafeteria Order Queue	Queue	Orders should be processed in the same order in which students place them, so a queue directly provides the required FIFO behavior.

11.3 Selected Feature for Implementation

Cafeteria Order Queue — chosen because it is the simplest and clearest real-world application of a Queue.

11.4 Implementation

```
class CafeteriaQueue:
    """Manage cafeteria orders using FIFO order."""

    def __init__(self):
        self.orders = []
```

```
def place_order(self, order):
    """Add a new order to the queue."""
    self.orders.append(order)

def serve_order(self):
    """Serve the oldest order."""
    if not self.orders:
        return None

    return self.orders.pop(0)

def display_orders(self):
    """Display all pending orders."""
    print("Pending Orders:", self.orders)
```

11.5 Test Code

```
cafeteria = CafeteriaQueue()

cafeteria.place_order("Order 101 - Burger")
cafeteria.place_order("Order 102 - Pizza")
cafeteria.place_order("Order 103 - Sandwich")

cafeteria.display_orders()

print("Serving:", cafeteria.serve_order())

cafeteria.display_orders()
```

11.6 Expected Output

```
Pending Orders: ['Order 101 - Burger', 'Order 102 - Pizza', 'Order 103 - Sandwich']
Serving: Order 101 - Burger
Pending Orders: ['Order 102 - Pizza', 'Order 103 - Sandwich']
```

11.7 Screenshots

```
1 class CafeteriaQueue:
2
3     def __init__(self):
4         self.orders = []
5
6     def place_order(self, order):
7         self.orders.append(order)
8
9     def serve_order(self):
10        if not self.orders:
11            return None
12
13        return self.orders.pop(0)
14
15    def display_orders(self):
16        print("Pending Orders:", self.orders)
17
18
19 cafeteria = CafeteriaQueue()
20
21 cafeteria.place_order("Order 101 - Burger")
22 cafeteria.place_order("Order 102 - Pizza")
23 cafeteria.place_order("Order 103 - Sandwich")
24
25 cafeteria.display_orders()
26
27 print("Serving:", cafeteria.serve_order())
28
29 cafeteria.display_orders()
```

```
PS D:\sairaj course> & "C:\Program Files\Python314\python.exe" "d:/sairaj course/Lab_11_Data_Structures.py"
Pending Orders: ['Order 101 - Burger', 'Order 102 - Pizza', 'Order 103 - Sandwich']
Serving: Order 101 - Burger
Pending Orders: ['Order 102 - Pizza', 'Order 103 - Sandwich']
PS D:\sairaj course>
```

12.TASK10—SMARTE-COMMERCEPLATFORM

12.1Objective

Select an appropriate data structure for each of five features of a Smart E-Commerce Platform, justify each choice in 2– 3 sentences, and implement one feature.

12.2Feature– Data Structure Selection

Feature	Data Structure	Justification
Shopping Cart Management	Deque	Products need to be added and removed dynamically, and a deque allows efficient insertion and removal from both ends.
Order Processing System	Queue	Orders should be processed in the same sequence in which customers place them, which a queue provides through FIFO processing.
Top-Selling Products Tracker	Priority Queue	Products can be prioritized according to their sales count, allowing the highest-selling products to be retrieved efficiently.
Product Search Engine	Hash Table	Product IDs are unique keys, making a hash table appropriate for fast product lookup.
Delivery Route Planning	Graph	Warehouses and delivery locations can be represented as vertices and the roads/routes between them as edges.

12.3Selected Feature for Implementation

Product Search Engine (using a Hash Table) — chosen because it is simple, practical, and clearly demonstrates why a Hash Table is useful.

12.4Implementation

```
class ProductSearch:
    """Product search system using a hash table."""

    def __init__(self):
        self.products = {}

    def add_product(self, product_id, product_name):
```

```
        """Add a product using its unique product ID."""
        self.products[product_id] = product_name

    def search_product(self, product_id):
        """Search for a product using its ID."""
        return self.products.get(product_id, "Product not found")

    def delete_product(self, product_id):
        """Delete a product using its ID."""
        if product_id in self.products:
            del self.products[product_id]
            return True

        return False
```

12.5 Test Code

```
store = ProductSearch()

store.add_product(101, "Laptop")
store.add_product(102, "Smartphone")
store.add_product(103, "Headphones")

print("Search 102:", store.search_product(102))

store.delete_product(102)

print("Search 102 after deletion:", store.search_product(102))
```

12.6 Expected Output

```
Search 102: Smartphone
Search 102 after deletion: Product not found
```

12.7 Screenshots

```
1 class ProductSearch:
2     def __init__(self):
3         self.products = {}
4
5     def add_product(self, product_id, product_name):
6         self.products[product_id] = product_name
7
8     def search_product(self, product_id):
9         return self.products.get(product_id, "Product not found")
10
11     def delete_product(self, product_id):
12         if product_id in self.products:
13             del self.products[product_id]
14             return True
15         return False
16
17
18
19 store = ProductSearch()
20
21 store.add_product(101, "Laptop")
22 store.add_product(102, "Smartphone")
23 store.add_product(103, "Headphones")
24
25 print("Search 102:", store.search_product(102))
26
27 store.delete_product(102)
28
29 print("Search 102 after deletion:", store.search_product(102))
```

```
PS D:\sairaj course> & "C:\Program Files\Python314\python.exe" "d:/sairaj course/Lab_11_Data_Structures.py"
Search 102: Smartphone
Search 102 after deletion: Product not found
PS D:\sairaj course>
```

13.OVERALL CONCLUSION

This lab covered the implementation of eight fundamental data structures — Stack, Queue, Linked List, Binary Search Tree, Hash Table, Graph, Priority Queue, and Deque — each generated and tested with the help of AI, along with proper docstrings explaining their behavior.

The final two tasks extended this understanding into real-world system design, requiring a suitable data structure to be chosen and justified for each feature of a Campus Resource Management System and a Smart E-Commerce Platform, with one feature from each fully implemented and tested. Overall, the lab reinforced not just how to implement each data structure, but why a particular structure is the right fit for a given operation, such as LIFO/FIFO ordering, fast key-based lookup, hierarchical search, or network-like relationships.